- [ ] Optional evidence: a team-defined lifecycle, reliability, memory, budget, provider, or coordination capability works as described.

1.11 Evaluation Criteria

This track will follow the evaluation criteria below:

Category	Weight	What reviewers will look for
End-to-end middleware behavior	**40%**	A real frontend-to-backend, Runtime, data, or infrastructure path with convincing functional evidence.
Technical design and integration	**25%**	A clear rationale, coherent architecture, appropriate boundary, focused changes, and extensible contracts.
Verification and robustness	**20%**	Automated tests, error handling, cleanup or recovery, redaction, and protection against obvious bypasses.
Demo and reproducibility	**15%**	A concise live demo, useful README, one-command startup, documented limitations, and no hidden manual setup.

1.12 Scope Guidance and Frequently Asked Questions

This is a hackathon-scale Agent infrastructure challenge, not a requirement to build a complete commercial cloud product. A strong submission may support one local Runtime path, a small mock resource set, and a focused middleware story. Depth, coherence, and relevance matter more than the number of example features implemented.

Teams are not required to train a foundation model, build a workflow editor, implement production OAuth, create a general-purpose sandbox, support multiple cloud regions, or deploy to ECS. Mock external services are acceptable, but static UI mockups cannot replace functional middleware behavior.

Do we need BytePlus ECS? No. Local Docker, Colima, or Podman is the default judging path. Cloud deployment is optional.

Do we have to select one recommended example? No. The examples are starting points. Teams may adapt, combine, simplify, replace, or invent capabilities that fit their platform.

Can we use mock users or resources? Yes. Controlled fixtures are encouraged when they make middleware behavior reproducible.

Does a polished UI count as middleware? No. The UI may explain and visualize a capability, but the behavior must execute in a trusted backend, Runtime, data, or infrastructure path.

Why does Ark return 401 Unauthorized? The most common cause is using a BytePlus account AK/SK instead of an Ark model API key, or using the wrong endpoint ID.

Where should we start reading the code? Begin with `apps/server/src/types.ts`, `apps/server/src/app.ts`, `apps/server/src/agent-service.ts`, and the two `AgentRunner` implementations. Then inspect `apps/web/src/App.tsx` for the smallest UI integration point.

How to access the Starter Kit? https://github.com/RrankPyramid/CodeJam

2. Autonomous Machine Learning Research Agent for Recommender Systems

⚠️ In response to some queries from our Early Bird participants, our engineers have provided updates to the problem statement to improve clarity and to support participants better.

Problem Statement last updated: 27 August 2026, 5:55PM.

Added downloadable kuairand-starter-kit.zip under 'Starter Kit'

Problem Statement in our Early Bird release doc is also the same version as is here.

✨ **Technical Workshop Webinar with Q&A** will be held on **28 Aug, 2:00 to 2:45pm.**

Click here to join the webinar!

2.1 Background

Motivation

Machine learning engineers (MLEs) spend much of their time on a single activity: **taking a dataset and a set of metrics, then iterating on a model again and again to push the score higher.** This work is inherently cyclic — every round repeats the same loop, shown in Figure 1.

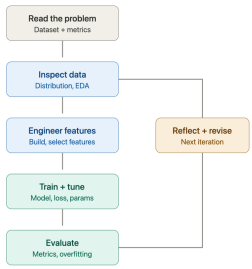


Figure 1. The MLE iteration loop

> **Figure 1. The MLE iteration loop.** A closed cycle of five core stages, plus a reflection step that feeds the next round:
>
> 1. **Read the problem** — understand the given dataset and the target metrics.
> 2. **Inspect data** — study data distribution through exploratory data analysis (EDA).
> 3. **Engineer features** — build and select input features (see Appendix A.5).
> 4. **Train + tune** — choose a model, set the loss function, and tune hyperparameters.
> 5. **Evaluate** — read the metrics, check for overfitting, and consult the leaderboard.
>
> The result of the **evaluate** stage drives a **reflect + revise** step, which decides what to change and loops back into the next iteration — re-inspecting the data and adjusting the features. The cycle repeats until the score plateaus.

Two of these stages — **engineer features** and **train + tune** — are carried out almost entirely in code: the engineer writes scripts to transform the data, define the model, and run training. In other words, each turn of the loop produces and modifies code. This is what makes the loop a natural target for automation: it is structured and repeatable, yet writing and revising that code is exactly the kind of task a code-generating LLM can take on.

The loop is also repetitive and mechanical. It draws heavily on "engineering intuition," but many individual steps are well-structured and repeatedly exercised in practice — which is precisely why automating the whole cycle has become an active research direction.

Prior Work

Over the past two years, a new line of work has set out to automate this loop: the **Autonomous ML Research Agent**, an LLM-driven agent that runs the cycle in Figure 1 on its own. It reads the problem, **writes the code** for each stage, trains and evaluates the model, reflects on the results, revises its approach, and finally produces a submission. Representative systems include:

- **MLE-Bench** [1] (OpenAI) — a benchmark of 75 Kaggle competitions, now a standard evaluation suite for such agents.

- **AIDE** [2] (Weco AI) — a state-of-the-art agent that frames ML engineering as code optimization and explores the space of solutions via tree search.
- **AI-Scientist-v2** [3] (Sakana AI) — an end-to-end agent for autonomous scientific and ML research, using agentic tree search to form hypotheses, run experiments, and write up results.

This Challenge

This challenge asks participants to design an **autonomous ML research agent**. Given a public ML dataset and a set of metrics, the agent must **autonomously** run the full loop of Figure 1 — read the problem, engineer features, train and tune the model, evaluate, then reflect and iterate — to reach the highest possible score across the test sets. Writing the code for each stage is part of the agent's job, not something provided in advance.

> **New to recommender systems?** All benchmarks in this challenge come from the recommendation domain (the KuaiRand family). If terms such as CTR, multi-task learning, GAUC, or NDCG are unfamiliar, start with the **Appendix: A Primer on Recommender Systems** . At the end of this document — a concept map plus an annotated reading list designed to get you oriented in 1–2 hours.

2.2 Problem Statement

The Task

Design and implement an Autonomous ML Research Agent. For each benchmark, the agent must autonomously:

1. **Reproduce the official baseline.** Stand up a working end-to-end pipeline and confirm it reaches the official baseline's reported validation score. (The official baseline is a fixed, organizer-provided reference — see *Benchmarks*. Any starter pipeline the agent builds for itself is an internal step, not the reference it is scored against.)
2. **Iterate on the pipeline.** Autonomously draw on established methods from both industry and academia to improve each stage of the pipeline (see Figure 1), and apply those improvements in code. The agent develops using **only the training split and the public validation feedback** — it never has access to the hidden test set.
3. **Improve over the baseline.** Through repeated iterations, drive the **validation** score above the official baseline. Improvement need not be strictly monotonic — as with real-world data, the trajectory may fluctuate — but the agent should show a clear, sustained ability to keep improving relative to the baseline. Final ranking is computed once, on the **hidden test set**, using the submission the agent designates as final.

Task Requirements

4. **Runs end-to-end and aims to beat the baseline.** The agent must run the full pipeline on the required benchmark (KuaiRand-Pure) and reach a converged result; attempting the bonus benchmark (KuaiRand-1k & KuaiRand-27k) is optional. The target is a hidden-test score that exceeds the official baseline; the actual delta achieved — positive or negative — is what feeds into the Primary metric scoring (see Judging Criteria), so falling short of the baseline is scored continuously rather than treated as a disqualifying failure.
5. **Iterates autonomously across the full stack.** The agent should improve the solution on its own, driven by its own evaluation of results. Improvements may target any part of the algorithmic stack — not just the model architecture, but every upstream and downstream module is fair game. The goal is to **minimize human intervention** — a fully autonomous run is the ideal, but a well-instrumented **semi-automated** pipeline that requires only a handful of interventions is an acceptable and realistic outcome; in practice, we measure how little human intervention a run requires (e.g. the number of manual interventions).
6. **Robust operation.** The pipeline should run reliably with **minimal human intervention**. Robustness here is about how the agent *handles* difficulty, not how often it succeeds — we do not score it by failure count, since a capable agent may fail only on genuinely hard problems. What matters is that when a step fails (a code error, a timeout, an unexpected input), the agent can recover, retry, or route around it, and that long iterative runs neither crash, stall, nor diverge.

2.3 Constraints & Scope

Category	Constraints & Scope Details
In scope	• Any open-source library or framework (PyTorch, RecBole, TorchRec, LightGBM, ...) • Any papers, public solutions, or pretrained weights • Changes to any pipeline stage — not just the model
Out of scope	• No external training data or pretrained weights trained on these benchmarks' test labels • No hidden-test access during development (train + validation only)
Limits	• **KuaiRand-Pure**: NDCG@10 / Recall@50, click = positive (fixed) (*Required*); **KuaiRand-1k** & **KuaiRand-27k**: same task and metrics (*Bonus*) • Hidden test scored once, on the final submission • **Compute budget**: **50 iterations** per benchmark run (hard cap; the convergence rule ε = 0.002 / N = 3 normally triggers first), plus a **6 h wall-clock** ceiling per run as a backstop. Compute is deliberately not the binding constraint on this benchmark — 100 iterations of the official baseline take about 28 min on a single CPU core with no GPU. GPU-hours and LLM tokens are reported for Feasibility scoring, not capped.
Allowed assumptions	• Fixed `train / validation / hidden-test` split per dataset • Official baseline, scores & evaluation script (incl. convergence rule) • Example submission + output schema

2.4 Available Resources & Data

Starter Kit

kuairand-starter-kit.zip
15.48KB

To lower the barrier to entry — especially for participants new to recommender systems — the challenge provides a standard starting point. **Download: kuairand-starter-kit.zip** (above) — numpy only (no torch / pandas / scikit-learn); `python3 baseline.py --model fm` reproduces the official baseline in about 40 s on a single CPU core. It contains:

1. **Fixed data splits**: date-based, taken from the two standard logs (`log_standard_4_08_to_4_21_pure.csv` & `log_standard_4_22_to_5_08_pure.csv`). **train** = date 20220408–20220421 (1,141,112 rows) / **validation** = date 20220422–20220428 (124,909 rows) / **test** = date 20220429–20220508 (170,588 rows). Teams develop on train + validation only; the hidden test set is scored once. Splitting by date rather than by row count avoids any tie-breaking ambiguity on equal timestamps.
2. **Official baseline**: a fixed, organizer-provided reference pipeline shipped in the Starter Kit — a Factorization Machine (k=16, lr=0.001, 5 categorical fields), numpy only, about 40 s on CPU. Published **hidden-test** scores: GAUC **0.6610** / nDCG@5 **0.5282** / primary **0.5946** (mean over 5 seeds, std 0.0008). Validation: GAUC 0.6674 / nDCG@5 0.5357 / primary 0.6016. Reference rungs for harness self-check — random scoring: primary 0.4753; item popularity: primary 0.5715. Beating *this* baseline is what counts — not a baseline the team builds itself.
3. **Evaluation script**: the exact scoring code (GAUC / nDCG@5) ships in the Starter Kit as `evaluate.py` . It is model-agnostic — it takes only `(user_ids, labels, scores)` , so any model can be scored with it. **Pinned conventions**: users with zero positives count as nDCG = 0 and are included in the average; GAUC counts only users with 0 < positives < impressions, weighted by positive count; nDCG gain = 2^rel − 1. **Convergence rule: ε = 0.002, N = 3** — a run is converged when the validation primary score has not improved by more than ε over the last N consecutive iterations (ε ≈ 2.5σ of the baseline's 5-seed std of 0.0008). The absolute-delta aggregation is unchanged.
4. **Submission format**: a CSV with the header `row_id,user_id,video_id,score` , one line per evaluation-split row. `row_id` is a 0-based, strictly increasing index into the split as produced by `data.load()` ; `user_id` / `video_id` are redundant fields used only to verify alignment; `score` is any real number (only the relative order matters), and NaN / Inf are rejected. The `row_id` is required because `(user_id, video_id)` is **not unique** in the evaluation split — 3.06% of test rows are repeated pairs, up to 12 times — so it cannot serve as a key. Generate a runnable example with `python3 submit.py --make` and validate with `--check` , which rejects a wrong header, a row-count mismatch, `row_id` gaps, misalignment against the evaluation split, and non-numeric scores.
5. **Run-log requirements**: each iteration should record its **hypothesis**, the **code diff**, the resulting **metrics**, and any **error / recovery events**. These logs are how judges assess **Autonomy** (scored under Impact & Relevance) and **Robustness** (scored under Technical Execution) — see Judging Criteria.
6. **LLM coding agent**: you can use whatever you like, or use Trae from ByteDance, which provides "Limited offer: new user 7-day free trial".

Benchmarks

KuaiRand-Pure is required and determines 100% of the primary score. **KuaiRand-1k and KuaiRand-27k are bonus datasets** — attempting them is optional and earns extra credit, but neither is required to complete the primary score.

Resource policy. This is a hackathon, so external resources are open by default: use any open-source library (PyTorch, RecBole, TorchRec, LightGBM, ...), read any papers, docs, or public solutions, and use pretrained model weights freely. The agent is expected to draw on whatever published methods it can find — that is what makes it a *research* agent.

There is **one hard rule: no external training data.** Training must rely only on the KuaiRand datasets listed below — no augmenting, joining, or pre-training on any other dataset, and no pretrained model whose weights were trained on these

benchmarks' test labels. This single rule is what keeps the hidden-test ranking fair; everything else is unrestricted.

Dataset	Domain & Description	Metrics	Scale
KuaiRand (Kuaishou) Three released variants: **KuaiRand-Pure** is required, while **KuaiRand-1k** and **KuaiRand-27k** are bonus.	Short-video feed. 12 feedback signals (click / like / follow / comment / forward / long_view / play_time ...) plus a randomized-exposure intervention that supports counterfactual evaluation. **Relevance label, task form and metrics are fixed by the organizers** (pinned in the Starter Kit): the task treats `long_view` (native column) as the positive relevance label, ranks **within each user's logged impressions** (not full-catalog retrieval), and reports **GAUC / nDCG@5**. Primary score = mean(GAUC, nDCG@5).	GAUC / nDCG@5	Pure: 1.4M interactions (27K users × 7.6K items). 1k: 11.7M. 27k: 322M.

Links: KuaiRand — https://kuairand.com

> KuaiRand's randomized-exposure data also enables off-policy / counterfactual evaluation (OPE).

2.5 Deliverables

1. **Written Project Description (via Devpost)**

- Provide a clear written description of your project that includes:
  - How your solution addresses the problem statement
  - Development tools used (e.g. VSCode, Colab, Jupyter)
  - APIs used (e.g. OpenAI GPT-4o, Google Maps API)
  - Libraries and frameworks used (e.g. Hugging Face Transformers, PyTorch, scikit-learn, pandas)
  - Datasets and assets used (e.g. Google Local Reviews dataset, manually labelled data)

2. **Public Code/GitHub Repository**

- Submit a link to a public Code/GitHub repository containing:
  - Well-structured, commented code covering all components of your solution
  - A README file that includes:
    - Project overview
    - Setup and installation instructions
    - Steps to reproduce your results
    - A brief reflection on your solution's limitations and what you would improve given more time
    - Team member contributions (if applicable, i.e. team participants, non-solo participants)

3. **Run & Iteration Logs**

- Submit the per-iteration log required in the Starter Kit (Run-log requirements), covering:
  - Hypothesis for that iteration — what the agent intended to try and why
  - The code diff applied
  - The resulting metrics (GAUC / nDCG@5 for the KuaiRand benchmarks)
  - Any error or recovery events encountered, and how the agent handled them
- A short summary reporting the number of manual interventions during the run (used to assess autonomy per Task Requirement 2)

4. **Final Submission & Results Summary**

- Submit your final model output/checkpoint for the required benchmark (KuaiRand-Pure), in the schema defined by the Starter Kit. If you also attempt the bonus benchmarks (KuaiRand-1k & KuaiRand-27k), submit their outputs as well for bonus scoring.
- A results table reporting your validation-best score for the required benchmark's metrics (KuaiRand-Pure GAUC / nDCG@5), and its absolute delta over the official baseline (per the Judging Criteria scoring formula); if you attempted the bonus benchmarks (KuaiRand-1k & KuaiRand-27k), include their GAUC / nDCG@5 results as well
- Reported resource usage required to reach the converged result: total token consumption (input + output) from the agent's LLM calls, the total **agent wall-clock** of the run, and the number of iterations used (out of the 50-iteration cap). Report GPU-hours as well if any GPU was used. These feed Feasibility & Practicality scoring.

2.6 Judging Criteria

Judging Criteria	Weight
Technical Execution	**35%**
Innovation & Problem Insight	**20%**
Impact & Relevance	**20%**
Feasibility & Practicality	**15%**
Presentation & Communication Final Event Only	**10%**

Technical Execution — Primary Metric & Robustness

Primary metric. We score the **converged result**, not the peak and not the intermediate trajectory. A run is considered converged when **validation score has not improved by more than ε = 0.002 over the last N = 3 consecutive iterations**, *or* when the run hits the **50-iteration cap** or the **6 h wall-clock ceiling** — whichever comes first. The submission scored for ranking is the **validation-best checkpoint** at that point, evaluated **once on the hidden test set**. The agent develops only on train + validation; it never sees the hidden test set.

- **KuaiRand-Pure is the required benchmark** and determines 100% of the Primary metric score. **KuaiRand-1k and KuaiRand-27k are bonus benchmarks**: a strong result on either earns additional bonus points on top of the Primary metric score, but skipping them does not reduce the KuaiRand-Pure score.
- Per-dataset metrics: **KuaiRand-Pure / KuaiRand-1k / KuaiRand-27k** → GAUC / nDCG@5. Within each dataset, the score is the **equal-weighted average of each metric's *absolute* improvement over the official baseline** on the hidden test set. For every metric *m*:

```
delta(m) = score_agent(m) − score_baseline(m)
score_dataset = mean over m of  delta(m)
```

- **Reading the numbers.** The metrics do not span [0, 1]. On the hidden test set, 27.1% of users have no positive label (their nDCG is 0 for any model) and 9.2% are all-positive, so a perfect ranking — using the true labels as the score — reaches only GAUC 1.0000 / nDCG@5 **0.7289** / primary **0.8645**. Random scoring sits at primary 0.4753. The official baseline's 0.5946 therefore already captures about 31% of the attainable range; judge progress against the 0.8645 ceiling, not against 1.0.

Robustness. Not judged by whether the agent ever hits a failure, but by **how it handles one** — recovering, retrying, or routing around a failed step (a code error, a timeout, an unexpected input) so that long iterative runs neither crash, stall, nor diverge before hitting the compute/wall-clock budget.

Innovation & Problem Insight

Judged on what the agent identified as worth trying and why — not on implementation.

- What the agent chose to target across the full algorithmic stack (features, model architecture, training strategy, evaluation loop, etc. — improvements are not limited to the model itself) and the reasoning behind that choice.
- Originality in drawing on published methods, papers, or public solutions — rewarding agents that go beyond naive baseline tweaks.

Impact & Relevance — Autonomy

Autonomy. How much of the improvement loop the agent drives on its own — proposing and testing changes based on its own evaluation of results, not just tuning the model architecture. Measured primarily by the **number of manual interventions** required to reach the converged result; fewer interventions score higher, with fully autonomous runs scoring highest. The fewer humans required, the more this reflects real acceleration of recommender-system R&D.

Feasibility & Practicality — Resource Consumption

How much it costs — in LLM usage and agent wall-clock — to reach the converged result. Two rules make this comparable: it is **scored only among submissions whose hidden-test primary score exceeds the official baseline**, and it is graded in **three coarse tiers** (low / medium / high consumption) rather than a continuous ranking. Without the quality gate the criterion would fight the Primary metric — an agent that stopped after three iterations would look cheapest and score worst.

- **Token consumption.** Total input + output tokens used by the agent's LLM calls across the run.
- **Agent wall-clock.** Total elapsed time of the agent run to reach the converged result. This replaces GPU-hours as the scored compute measure: on this benchmark the reference pipeline needs no GPU at all (about 28 min of single-core CPU for 100 iterations), so GPU-hours would be ~0 for most teams and would only penalise whoever happened to use a GPU. Report GPU-hours if any were used, but wall-clock is what is scored.

2.7 References

[1] J. S. Chan, N. Chowdhury, O. Jaffe, J. Aung, D. Sherburn, E. Mays, G. Starace, K. Liu, L. Maksin, T. Patwardhan, L. Weng, and A. Mądry, "MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering," OpenAI, 2024. arXiv:2410.07095. https://doi.org/10.48550/arXiv.2410.07095

[2] Z. Jiang, D. Schmidt, D. Srikanth, D. Xu, I. Kaplan, D. Jacenko, and Y. Wu, "AIDE: AI-Driven Exploration in the Space of Code," 2025. arXiv:2502.13138. https://doi.org/10.48550/arXiv.2502.13138

[3] Y. Yamada, R. T. Lange, C. Lu, S. Hu, C. Lu, J. Foerster, J. Clune, and D. Ha, "The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search," 2025. arXiv:2504.08066. https://doi.org/10.48550/arXiv.2504.08066

[4] H. Zhao, G. Cai, J. Zhu, Z. Dong, J. Xu, and J.-R. Wen, "Counteracting Duration Bias in Video Recommendation via Counterfactual Watch Time," KDD 2024. Code: https://github.com/hyz20/CWM — *optional advanced reference, not the official baseline.* Its contribution is a censored-regression loss on watch time (a completed play means the true watch time was truncated by video length, so a one-sided loss is used instead of squared error). Note it ships no Recall implementation, reports nDCG@1/3/5 on a rebuilt `long_view2` label, and requires `torch==1.6.0`.

2.8 Appendix A. A Primer on Recommender Systems

> This appendix gives participants without a recommender-systems background just enough to get started. It is a concept map plus an annotated reading list — not a textbook. Use it to understand the KuaiRand benchmarks and to know what to look up when you get stuck.

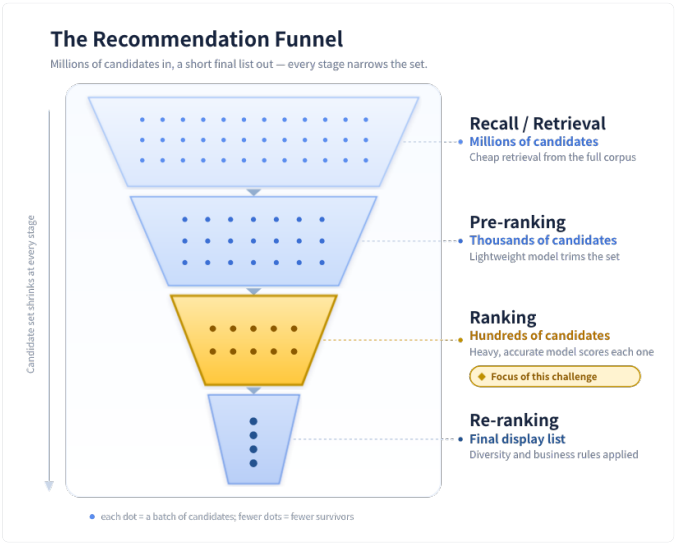
A.1 The Big Picture: The Recommendation Pipeline

A modern industrial recommender does not score every item directly. It runs a funnel of stages, each narrowing the candidate set:

```
Recall  →  Pre-ranking  →  Ranking  →  Re-ranking
millions    thousands       hundreds     final list
```

- **Recall / Retrieval**: cheaply retrieve a few thousand candidates from millions.
- **Pre-ranking**: a lightweight model trims the candidates further.
- **Ranking**: a heavy, accurate model scores each candidate. **This challenge mostly lives here.**
- **Reranking**: adjust the final ordering for diversity, business rules, and so on.

> For this competition you mainly need the **ranking** stage. The KuaiRand benchmarks are ranking/prediction tasks, not full end-to-end pipelines.



A.2 Core Tasks: CTR and the Feedback Funnel

Most industrial ranking is framed as predicting the probability of user feedback:

- **CTR (Click-Through Rate)** — `P(click | impression)`. The user saw the item; will they click?

- **CVR (Conversion Rate)** — `P(conversion | click)`. The user clicked; will they convert (buy)? E-commerce background only; not a task in this challenge.
- **The funnel**: `impression → click → deeper engagement` (in e-commerce, `→ conversion`). Because these stages are linked, two well-known problems arise:
  - **Sample selection bias**: the post-click signal is only observed on *clicked* items, yet must be predicted for *all* impressions.
  - **Data sparsity**: post-click signals such as `long_view` or `like` are far rarer than clicks.

> **KuaiRand** has no purchase label, so CVR itself is never scored here. The funnel framing above is general background — note that in KuaiRand the scored label `long_view` is logged on *every* impression, not only on clicked ones, so classic sample selection bias does not apply directly to this challenge's task. Data sparsity still does, and the multi-feedback structure (`click`, `like`, `follow`, `play_time` ...) makes ESMM-style multi-task modelling — see A.3 — a legitimate way to exploit the other signals as auxiliary tasks.

A.3 Multi-Task & Multi-Feedback Learning

Real users produce many signals (click, like, follow, comment, watch-time, and so on). Predicting them jointly — rather than training a separate model per signal — shares representations and tends to improve every task.

- Why it matters here: **KuaiRand** provides **12 feedback signals**, so a multi-task model can learn from several of them jointly even though only `long_view` is scored.
- The key idea is to balance *shared* parameters (which transfer useful knowledge across tasks) against *task-specific* parameters (which prevent conflicting tasks from hurting one another — the "seesaw" problem).

A.4 Evaluation Metrics

Metric	Intuition	Used for
AUC	Probability that a random positive is ranked above a random negative. Threshold-free and robust to class imbalance.	**Scored in this challenge** as **GAUC** — per-user AUC averaged with each user's positive count as the weight; users whose impressions are all-positive or all-negative are excluded.
NDCG	Quality of a *ranked list*, rewarding relevant items near the top (with a position discount).	**Scored in this challenge** as **nDCG@5**. Users with no positive label score 0 and are included in the average.
Recall	Fraction of all relevant items that appear in the returned list.	Retrieval / coverage tasks — **not scored here.** Each user has only ~5 logged impressions in the evaluation split, so Recall@50 is 0.999+ for every model, including random scoring.

> **Offline vs. online**: a higher offline metric does not always mean better real-world performance (because of distribution shift and feedback loops). This competition is evaluated offline, but it is worth knowing the gap exists.

A.5 Feature Engineering Basics

- **ID features**: user ID, item ID, category ID — high-cardinality discrete features.
- **Embedding**: map each discrete ID to a learnable dense vector. This is the foundation of all deep recommenders.
- **Feature crossing**: combine features (e.g. user × category) to capture interactions. Models such as FM and DeepFM automate this.

A.6 Annotated Reading List

[Hints: If you find reading the following material challenging or find you have missing backgrounds, you can use ChatGPT / Claude / ... to explain it to you.]

The goal here is only to understand **how a recommender system is structured** — the recall → ranking → re-ranking pipeline — and where the ranking stage (which this challenge targets) sits within it. You do **not** need to read a whole course; the introductory overview is enough. **Read just one of the following:**

- Google, *Recommendation Systems* (Machine Learning Crash Course), the **Overview** section — `https://developers.google.com/machine-learning/recommendation` A short, official overview of the pipeline. Note: Google calls the ranking stage **"scoring"** — this is the same thing as **ranking**, and it is the part this challenge focuses on.
- Wang Shusen, *Recommender Systems*, **Chapter 1 (Overview)** — `https://github.com/wangshusen/RecommenderSystem` The most beginner-friendly Chinese resource; the first chapter alone gives the full architecture.

3. Implement a GPU Kernel for a Transformer Layer

⚠️ In response to some queries from our Early Bird participants, our engineers have provided updates to the problem statement to improve clarity and to support participants better.

Problem Statement last updated: 27 August 2026, 6:25PM

Added Appendix: Test Shapes

Updated torch_transformer_benchmark.py

✨ **Technical Workshop Webinar with Q&A** will be held on **28 Aug, 3:00 to 3:45pm.**

Click here to join the webinar!

3.1 Background

Transformer is a widely used neural network architecture in modern AI. It is the core structure behind many natural language processing, computer vision, speech, recommendation, and large language model systems.

The main idea of Transformer is **self-attention**. Self-attention allows each token in a sequence to interact with other tokens directly. Compared with recurrent models, Transformer can process tokens in parallel, which makes it suitable for GPU acceleration.

Given an input sequence represented as a matrix:

$$X \in \mathbb{R}^{N \times d}$$

where N is the sequence length and d is the hidden dimension, the Transformer first projects the input into Query, Key, and Value matrices:

$$Q = XW_Q$$
$$K = XW_K$$
$$V = XW_V$$

The scaled dot-product attention is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where d_k is the dimension of each attention head. The scaling factor $\sqrt{d_k}$ is used to prevent the dot-product values from becoming too large, which could make the softmax distribution unstable.

However, the computation of Transformer is expensive. Important operations include matrix multiplication, attention score calculation, softmax, normalization, and feed-forward layers. These operations may be limited by GPU compute throughput, memory bandwidth, cache efficiency, kernel launch overhead, and tensor core utilization.

In this competition, participants are asked to use AI-assisted methods to optimize the runtime efficiency of a Transformer structure on a given GPU model. The optimized implementation should improve performance while keeping the output numerically correct compared with the reference implementation.

Participants may consider optimization methods such as operator fusion, memory layout optimization, reduced-precision computation, tensor core usage, softmax optimization, and custom CUDA, Triton, TensorFlow, or PyTorch implementations.